

## Practical 10 :- Asynchronous Processing with Event-Driven Architecture

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objective:</b>         | To implement asynchronous background processing using Node.js native EventEmitter without external dependencies.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Problem Statement:</b> | <ul style="list-style-type: none"> <li>• Add a background notification system to the Task Management application using Node.js built-in EventEmitter.</li> <li>• When a new task is created via the API, emit a task-created event handled asynchronously.</li> <li>• The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response.</li> <li>• The main API response must return immediately, before the event handler finishes executing.</li> <li>• Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Implementation</b>     | <p>Tasklistener.js</p> <pre>import taskEvents from './taskEvents.js';  /**  * Background Event Listeners.  *  * Registered once at application startup. Handles asynchronous, non-  blocking  * side effects triggered by Task lifecycle events.  */  // Listener 1: Task Created -&gt; Async Notification &amp; Email Dispatch taskEvents.on('task-created', (task) =&gt; {   const handlerStartTime = new Date().toISOString();   console.log(`[Event:Received] 'task-created' event caught at \${handlerStartTime}`);    // Supplementary Problem 3: Simulate heavy background notification (e.g. SMTP / Webhook)   // Using setTimeout proves non-blocking execution while the API response has already returned.   setTimeout(() =&gt; {     const completionTime = new Date().toISOString();     console.log('----- ');     console.log(`[Notification Service] Email dispatched to: \${task.user    'Workspace Member'}`);     console.log(`[Notification Details] Task: "\${task.title}" (Priority: \${task.priority    'medium'})`);     console.log(`[Notification Time]    Completed at: \${completionTime}`);     console.log('----- ');   }, 1500); // 1.5 second artificial delay simulating third-party email API });  // Listener 2 (Supplementary Problem 1): Task Deleted -&gt; Audit Log Record taskEvents.on('task-deleted', (data) =&gt; {   const timestamp = new Date().toISOString();   console.log('-----');   console.log(`[Audit Trail] Task ID "\${data.id}" permanently deleted`);</pre> |

```
console.log(`[Audit Action] Triggered by: ${data.user || 'Authenticated User'}`);
console.log(`[Audit Time]    Logged at: ${timestamp}`);
console.log('-----');
});

// Listener 3 (Supplementary Problem 2): Error Listener
// Catches and logs errors thrown inside event handlers, preventing Node
// process crashes.
taskEvents.on('error', (err) => {
  console.error('[EventEmitter:Error] Unhandled error in background
listener:', err.message);
});

console.log('[EventEmitter] Background task listeners successfully
registered');
```

Taskevents.js

```
import { EventEmitter } from 'events';

/**
 * TaskEvents – Application-wide Event Bus.
 *
 * Uses Node.js native EventEmitter without external message brokers.
 * Decouples resource mutation (API response) from downstream side-effects
 * (email notifications, audit logging, analytics indexing).
 */
class TaskEventEmitter extends EventEmitter {
  constructor() {
    super();
    // Default max listeners threshold
    this.setMaxListeners(20);
  }
}

export const taskEvents = new TaskEventEmitter();

export default taskEvents;
```

|         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         | <pre>PS D:\SEM 5\ANDF\task-manager-api&gt; cd .\backend\<br/><br/>&gt;&gt;&gt; [API Response] HTTP 201 Created sent to client at: 2026-10-08T12:27:24.466Z (24 ms elapsed)<br/>[API Controller] Emitting "task-created" event onto native EventEmitter bus...<br/>[Event:Received] 'task-created' event caught at 2026-10-08T12:27:24.466Z<br/><br/>&gt;&gt;&gt; Notice: Client is already released! Background work continues asynchronously...<br/><br/>-----<br/>[Notification Service] Email dispatched to: student@taskflow.dev<br/>[Notification Details] Task: "Deploy Production Release v2.4" (Priority: high)<br/>[Notification Time]    Completed at: 2026-10-08T12:27:25.976Z<br/>-----<br/><br/>=====</pre> <p>Demonstrating Supplementary Problem 1: task-deleted Event</p> <pre>=====</pre> <pre>&gt;&gt;&gt; [API Response] HTTP 200 OK sent to client at: 2026-10-08T12:27:26.484Z<br/>-----<br/>[Audit Trail] Task ID "66fc9876543210fedcba4321" permanently deleted<br/>[Audit Action] Triggered by: student@taskflow.dev<br/>[Audit Time]   Logged at: 2026-10-08T12:27:26.485Z<br/>-----<br/><br/>=====</pre> <p>Demonstrating Supplementary Problem 2: error Event Listener</p> <pre>=====</pre> <pre>[EventEmitter:Error] Unhandled error in background listener: Simulated external notification API timeout<br/>[Process] Event loop verified non-blocking. Demo completed successfully.</pre> |
| Outcome | Students will be able to implement non-blocking, event-driven background processing using Node's built-in EventEmitter and explain how it keeps the main request-response cycle fast.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |